

Architecture ETL IOL

Kafka transporte les donnees normales, RustFS transporte le Big Data - 27 juillet 2026

Decision centrale : api-core est le seul composant autorise a ouvrir la source. Hop et Spark recoivent un artefact verifie, jamais la connexion source.

Deux chemins automatiques

- Charge normale : Source -> api-core -> Kafka (toutes les donnees) -> consumer -> Hop -> destination.
- Big Data : Source -> api-core -> RustFS ; Kafka transporte le manifeste ; consumer -> Spark -> destination.
- JDBC est serialise en JSON type, jamais en CSV.
- Une ligne trop grande pour Kafka bascule automatiquement vers RustFS.

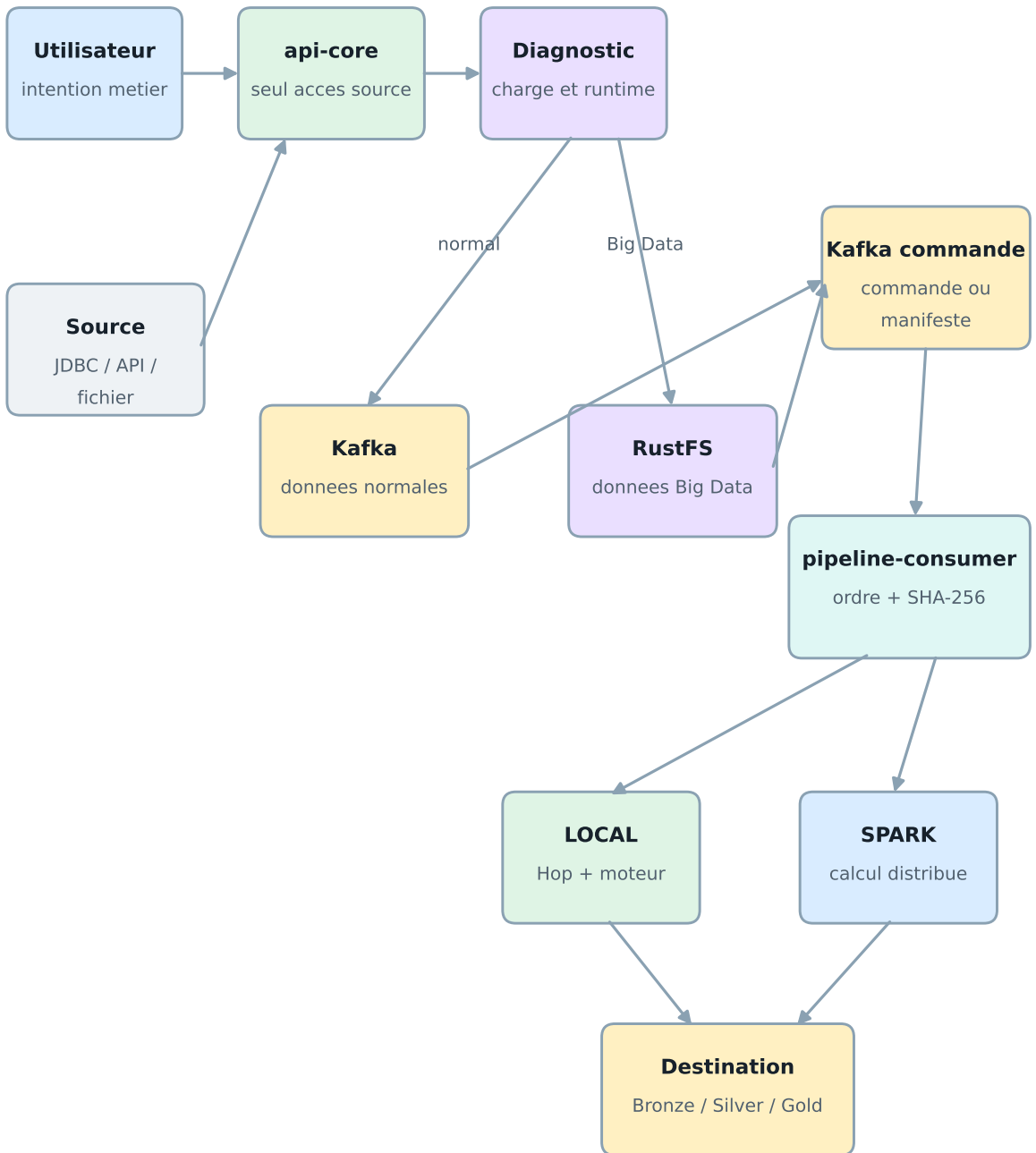
Ce que voit l'utilisateur

- La source, la destination, les colonnes, les regles Silver/Gold et l'etat du flux.
- Aucun choix Hop, Spark, Kafka ou RustFS dans le parcours metier.
- Les details techniques restent dans l'administration, les logs et le monitoring.

A retenir

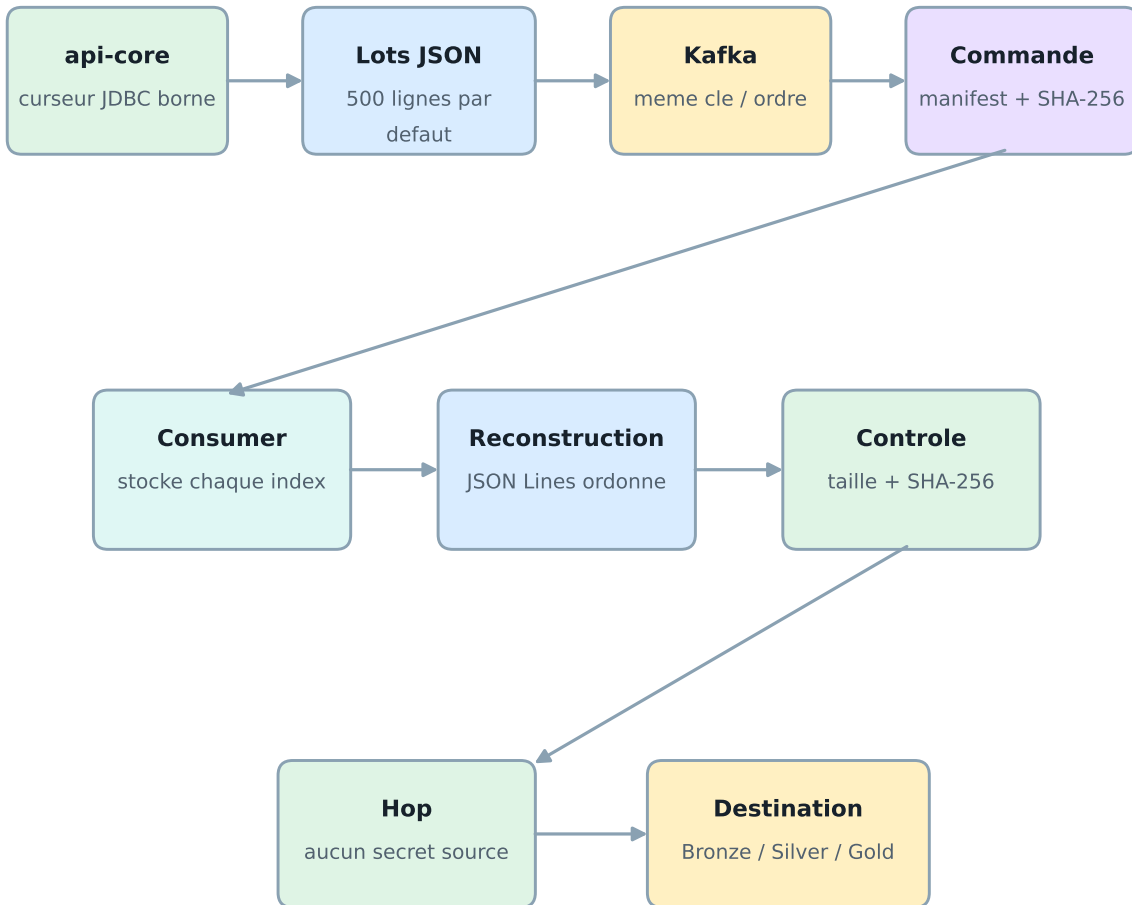
La plateforme choisit le transport et le runtime. L'utilisateur choisit le resultat attendu.

Vue generale



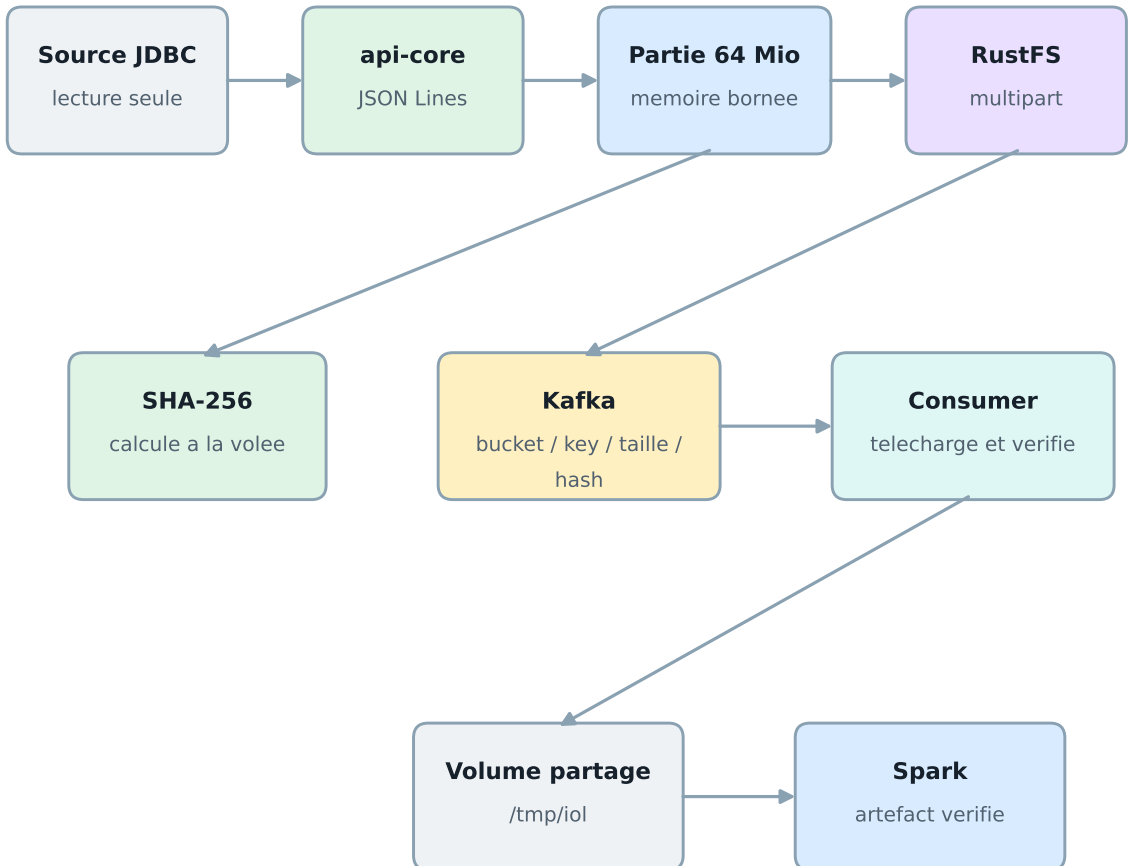
Aucune fleche ne relie la source a Hop ou Spark. La frontiere d'accès source s'arrete a api-core.

Charge normale : toutes les donnees dans Kafka



- Les types JSON sont conserves : null, nombre, booleen, texte et valeurs temporelles.
- Un doublon identique est idempotent ; un doublon different est refuse.
- Un lot manquant ou un hash invalide empeche seulement cette execution de demarrer.
- JDBC n'est jamais materialise en CSV.

Big Data : streaming multipart vers RustFS



- Pas de fichier JDBC complet sur le disque de api-core.
- Le multipart est annule en cas d'erreur avant finalisation.
- Kafka ne transporte que le manifeste lorsque la charge est Big Data.
- Succes : statut terminal, ACK Kafka, puis suppression immediate de l'objet RustFS.
- Echec : conservation 72 h maximum, puis purge planifiee toutes les heures.

Barriere de securite avant Hop et Spark



Le consumer refuse l'execution si

- une source non-PUSH n'a pas ete reconstruite depuis Kafka ou RustFS ;
- le protocole courant est POSTGRES, MYSQL, ORACLE, MSSQL, SQLITE ou un autre JDBC ;
- source_connection, source_connection_id, query, username ou password source subsiste ;
- la taille ou le SHA-256 ne correspond pas au manifeste.

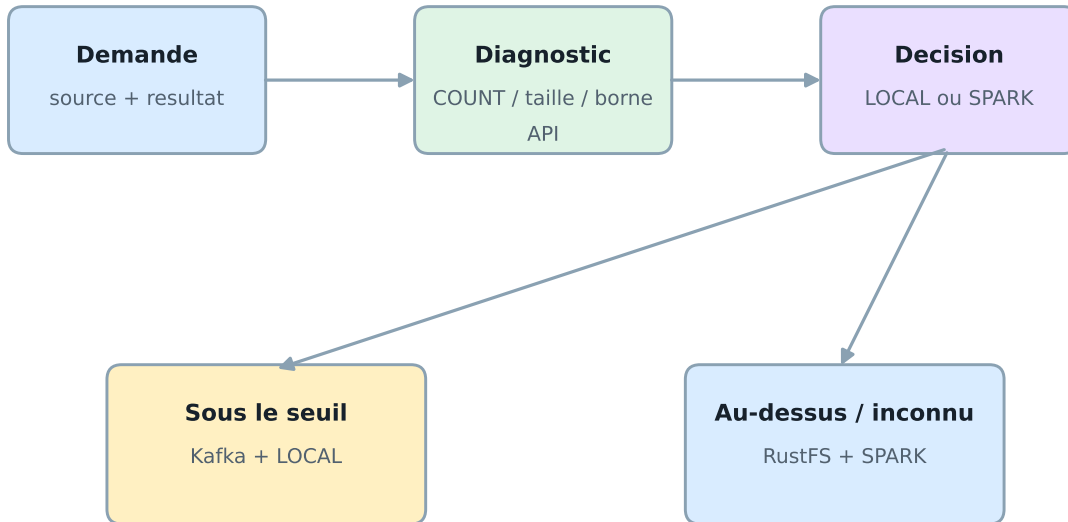
Ce qui reste autorise

target_connection reste presente : Hop ou Spark doit ecrire dans la destination. Cette connexion cible n'autorise pas un retour vers la source.

A retenir

Les moteurs contiennent aussi une defense en profondeur : un protocole JDBC source provoque une erreur explicite.

Decision automatique et experience utilisateur

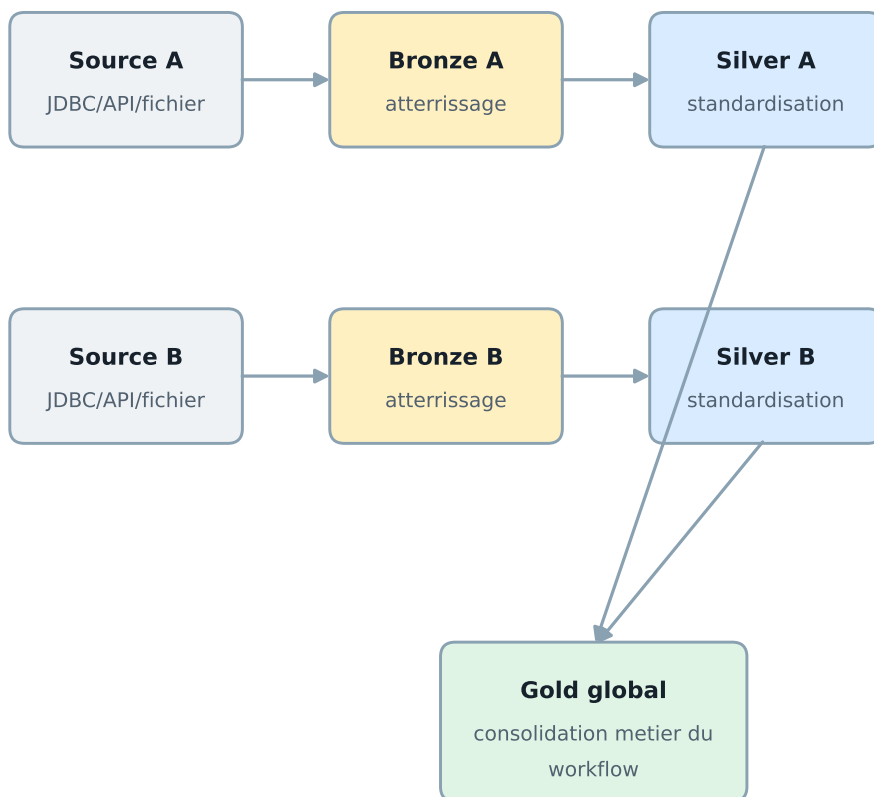


- Seuil JDBC de production : 10 000 000 lignes.
- Seuil fichier de production : 2 Gio.
- Un diagnostic JDBC incertain choisit SPARK par prudence.
- Un gros fichier peut declencher SPARK selon sa taille.
- Le choix manuel transport_mode ne force plus RustFS sur une petite charge.
- loadAssessment conserve la raison de la decision pour l'audit.

Regle UX

L'utilisateur metier ne choisit aucun composant technique. Les termes Hop, Spark, Kafka et RustFS restent dans les vues admin, l'exploitation et les logs.

Bronze, Silver, Gold



Interpretation

- Bronze garde une trace proche de la source : projection, nettoyage technique des noms de colonnes, batch et watermark.
- Silver est par source : typage, nettoyage, normalisation, regles de qualite.
- Gold est au niveau workflow : il combine les Silver pour produire la table metier finale.

Ancrages dans le code et controles production

Code

- SourceLoadEstimatorService.java : diagnostic automatique de charge.
- KafkaPipelineEventService.java : choix final LOCAL/SPARK.
- SourceDataTransportService.java : JDBC vers Kafka JSON ou RustFS.
- ObjectStorageService.java : multipart streaming et SHA-256.
- KafkaDataChunkStore.java : ordre, doublons et controle d'integrite.
- PipelineOrchestrator.java : barriere anti-acces source.
- moteur_universel.py et spark_etl.py : JDBC source explicitement interdit.

Production

- Activer TLS/SASL et les ACL Kafka.
- Utiliser des credentials RustFS uniques et une rotation des secrets.
- Limiter le compte source a SELECT et le compte cible aux schemas requis.
- Configurer la retention Kafka et le cycle de vie RustFS.
- Superviser les erreurs de hash, lots manquants, multipart abandonnes et espace /tmp/iol.
- Calibrer les seuils avec les ressources et les volumes reels.

A retenir

Resultat : le transport est coherent, observable et independant du moteur de calcul choisi.